DARIN'S HANDMADE — полный разбор кода

Подготовка к защите проекта. Здесь разобрано, откуда берётся каждая функция, что делает каждая библиотека и какой файл с каким общается.

Как читать: часть 1 — общая картина и библиотеки. Часть 2 — основы JavaScript, без которых код не понять. Части 3 и 4 — бэкенд и фронтенд по файлам, построчно. Часть 5 — полные цепочки «нажал кнопку → что произошло». Часть 6 — вопросы, которые могут задать, с ответами. Часть 7 — слабые места, о которых лучше знать заранее.

Часть 1. Общая картина

1.1 Из чего состоит проект

Проект — это два отдельных приложения в одной папке:

```
DarinsHandMade/
├── back/    сервер: Node.js + Express + MongoDB (через Mongoose)
├── front/   сайт:   React + Vite + Tailwind
└── docs/    документация
```

Они общаются только через HTTP-запросы. Фронт отправляет запрос на адрес вида `/api/...`, сервер отвечает JSON. Фронт никогда не ходит в базу данных напрямую — только сервер.

```
Браузер (front)  ──HTTP запрос──▶  Сервер (back)  ──▶  MongoDB Atlas (база)
                 ◀──JSON ответ──                  ──▶  Stripe (оплата)
                                                  ──▶  Resend (письма)
                                ◀──webhook──────────  Stripe
```

1.2 Откуда берутся библиотеки

Каждая библиотека, которую ты видишь в `require(...)` или `import ... from`, записана в файле `package.json`. Команда `npm install` читает этот файл, скачивает библиотеки из интернета (с сайта npmjs.com) и кладёт в папку `node_modules`. Когда код пишет `require('bcryptjs')`, Node ищет папку `node_modules/bcryptjs` и загружает её.

Правило, которое нужно знать на защите:

- `require('bcryptjs')` — **без** `./` в начале → библиотека из `node_modules` (чужой код).
- `require('./db/connect')` или `require('../models/userModel.js')` — **с** `./` или `../` → наш собственный файл из проекта.
- `require('crypto')`, `require('path')`, `require('fs')` — встроенные модули самого Node.js. Их нет в `package.json`, они идут вместе с Node.

Библиотеки сервера (back/package.json)

Библиотека	Зачем она	Где используется
`express`	веб-сервер: принимает HTTP-запросы и отправляет ответы	`index.js`, все `routes/`
`mongoose`	работа с MongoDB: схемы, модели, запросы `findOne`, `create` и т.д.	`models/`, контроллеры, `db/connect.js`
`bcryptjs`	хеширование паролей	`AuthController.js`
`jsonwebtoken`	создание и проверка JWT-токенов	`AuthController.js`, `authMiddleware.js`, `OrderController.js`
`cors`	разрешает сайту с другого адреса обращаться к серверу	`index.js`

`helmet`	добавляет защитные HTTP-заголовки	`index.js`
`express-rate-limit`	ограничивает число запросов с одного IP	`index.js`
`dotenv`	читает файл `.env` в `process.env`	`index.js`, `seed/`
`multer`	принимает файлы (картинки) из формы	`middleware/upload.js`
`stripe`	оплата картой	`OrderController.js`
`resend`	отправка писем	`utils/email.js`
`nodemon`	(только для разработки) перезапускает сервер при сохранении файла	команда `npm run dev`
`express-mongo-sanitize`, `xss-clean`	установлены, но **в коде не подключены**	нигде (см. часть 7)

Встроенные модули Node: `crypto` (случайные байты и хеши SHA-256), `path` (склейка путей к файлам), `fs` (работа с диском: создать папку, удалить файл).

Библиотеки фронта (front/package.json)

Библиотека	Зачем она
`react`, `react-dom`	сама библиотека интерфейса и вывод её в браузер
`react-router-dom`	переходы между страницами без перезагрузки (`Routes`, `Route`, `Link`, `useNavigate`, `useParams`)
`axios`	отправка HTTP-запросов на сервер
`tailwindcss` + `@tailwindcss/vite`	стили через классы прямо в разметке (`bg-black`, `text-[18px]`)
`lucide-react`	готовые иконки (корзина, пользователь, мусорка)
`framer-motion`	анимации
`react-image-magnifier-lens`	лупа для увеличения фото
`vite`	сборщик: запускает сайт в разработке (`npm run dev`) и собирает для публикации (`npm run build`)
`eslint`	проверка кода на ошибки стиля

1.3 Переменные окружения (.env)

Секреты не пишутся в код. Они лежат в файле `back/.env`, который не попадает в git. Библиотека `dotenv` при старте читает этот файл, и значения становятся доступны как `process.env.ИМЯ`.

Переменная	Для чего
`MONGO_URI`	адрес базы MongoDB Atlas с логином и паролем
`JWT_SECRET`	секрет для подписи токенов. Кто его знает — может подделать токен
`PORT`	порт сервера (по умолчанию 5000)

`CLIENT_URL`	адрес сайта, чтобы строить ссылки в письмах
`STRIPE_SECRET_KEY`	ключ Stripe для создания оплаты
`STRIPE_WEBHOOK_SECRET`	секрет для проверки, что вебхук пришёл именно от Stripe
`RESEND_API_KEY`	ключ сервиса писем
`MAIL_FROM`	от чьего имени уходят письма
`MANAGER_EMAIL`	куда отправлять отчёт о новом заказе

На фронте есть одна переменная — `VITE_API_URL` (адрес сервера после публикации). Vite отдаёт её коду как `import.meta.env.VITE_API_URL`.

1.4 Архитектура сервера: три слоя + middleware

```
запрос → index.js → routes/ → middleware/ → controllers/ → models/ → MongoDB
```

- **routes/** — какой адрес какую функцию вызывает. Никакой логики.
- **middleware/** — проверки, которые выполняются до контроллера: токен, роль, файл, валидация.
- **controllers/** — сама логика: проверить данные, обратиться к базе, отправить ответ.
- **models/** — описание того, как выглядят данные в базе (схемы).

Зачем так: каждый файл отвечает за одно. Если поменять правило проверки пароля, трогаешь только `validate.js`, а не все контроллеры.

Часть 2. Основы JavaScript, которые встречаются в коде

Этот раздел объясняет синтаксис. Если на защите спросят «а что это за запись?» — ответ здесь.

2.1 require и module.exports (сервер)

Сервер использует систему модулей CommonJS.

```
// файл userModel.js — отдаёт наружу
module.exports = User;

// файл AuthController.js — забирает
const User = require('../models/userModel.js');
```

`module.exports` — то, что файл отдаёт. `require` — получить это в другом файле. Если файл отдаёт объект с несколькими функциями:

```
module.exports = { signup, signin, getMe };          // AuthController.js
const { signup, signin, getMe } = require('../controllers/AuthController'); // authRoute.js
```

2.2 import и export (фронт)

Фронт использует ES-модули — современный вариант того же самого.

```
export default App;                       // один главный экспорт файла
import App from './App';                  // забрать его (имя можно любое)

export const imageUrl = ...;              // именованный экспорт
import api, { imageUrl, authHeader } from './api';  // главный + именованные в фигурных скобках
```

2.3 Деструктуризация

Достать поля из объекта в отдельные переменные одной строкой.

```
const { email, password } = req.body;
// то же самое, что:
const email = req.body.email;
const password = req.body.password;
```

С массивом так же, только по порядку: `const [token, setToken] = useState(...)` — первый элемент и второй.

2.4 Стрелочные функции

```
const hashToken = (raw) => crypto.createHash('sha256').update(raw).digest('hex');
// то же самое, что:
function hashToken(raw) {
  return crypto.createHash('sha256').update(raw).digest('hex');
}
```

Если после `=>` нет фигурных скобок — результат возвращается автоматически.

2.5 async / await и Promise

Запрос к базе занимает время. JavaScript не ждёт, а сразу идёт дальше и получает **Promise** — «обещание», что результат будет позже.

```
const user = await User.findOne({ email });
```

`await` говорит: «остановись здесь, пока база не ответит, и положи ответ в `user`». `await` можно писать только внутри функции, помеченной `async`. Поэтому все контроллеры объявлены как `const signin = async (req, res) => { ... }`.

На фронте тот же Promise обрабатывается через `.then()` и `.catch()`:

```
api.post(url, body)
  .then((response) => { /* сервер ответил успешно (код 2xx) */ })
  .catch((error)    => { /* ошибка: сеть или код 4xx/5xx */ });
```

2.6 try / catch

```
try {
  // код, который может упасть (например, база недоступна)
} catch (error) {
  // сюда попадаем, если внутри try что-то бросило ошибку
  res.status(500).send({ message: error.message });
}
```

Без `try/catch` ошибка в контроллере могла бы оставить клиента без ответа.

2.7 Опциональная цепочка ?. и оператор ||

```
error.response?.data?.message || "Server error"
```

`?.` — «если слева `null` или `undefined`, не падай, а верни `undefined`». `||` — «если слева пусто, возьми значение справа». Вместе: взять сообщение сервера, а если его нет — написать «Server error».

`??` (в `p.price ?? ""`) похож на `||`, но заменяет только `null`/`undefined`, а `0` оставляет. Для цены и остатка это важно: `0` — нормальное значение.

2.8 Шаблонные строки

```
`${base}/reset-password?token=${rawToken}`
```

Обратные кавычки и `${...}` — вставить значение переменной прямо в строку.

2.9 Spread ...

```
setForm({ ...form, [e.target.name]: e.target.value });
```

`...form` копирует все поля старого объекта в новый, а потом одно поле перезаписывается. `[e.target.name]` — имя поля берётся из переменной (если поле называется `email`, получится `email: значение`).

2.10 HTTP: методы и коды ответа

Метод	Смысл	Пример в проекте
GET	получить	`GET /api/products`
POST	создать / отправить	`POST /api/auth/signin`
PUT	заменить / обновить целиком	`PUT /api/products/:id`
PATCH	обновить часть	`PATCH /api/orders/:id/status`
DELETE	удалить	`DELETE /api/cart/:productId`

Код	Значение	Когда у нас
200	OK	успешный ответ
201	Created	создан пользователь или товар
400	Bad Request	неверные данные, неверный пароль
401	Unauthorized	нет токена или он недействителен
403	Forbidden	токен есть, но роль не та
404	Not Found	товар или заказ не найден
500	Server Error	что-то упало на сервере
501	Not Implemented	оплата не настроена (нет ключа Stripe)

Часть 3. Сервер (back/) по файлам

3.1 back/index.js — точка входа

Этот файл запускается командой `node index.js` (или `npm run dev`). Он собирает сервер из частей.

```
const express = require('express');
const app = express();
```

`express` — функция из библиотеки. Её вызов создаёт объект приложения `app`. У `app` есть методы:

- `app.use(...)` — подключить middleware или роутер для всех (или части) адресов;
- `app.post(...)`, `app.get(...)` — обработчик для конкретного метода и адреса;
- `app.set(...)` — настройка;
- `app.listen(port)` — начать принимать запросы.

Порядок строк в index.js важен. Express проходит по ним сверху вниз для каждого запроса.

1. `app.set('trust proxy', 1)` — сервер на хостинге стоит за прокси. Без этого Express видит IP прокси вместо IP посетителя, и лимит запросов считал бы всех посетителей за одного человека.
2. `app.use(cors())` — `cors` из библиотеки `cors`. Браузер по умолчанию запрещает сайту с одного адреса (фронт) обращаться к другому (сервер). `cors()` добавляет заголовок, который это разрешает.
3. `app.post('/api/orders/webhook', express.raw({ type: 'application/json' }), stripeWebhook)` — **стоит до** `express.json()`. Stripe подписывает сырые байты тела запроса. Если сначала превратить тело в объект, исходные байты потеряются и подпись не сойдётся. `express.raw` оставляет тело как есть (Buffer).
4. `app.use(express.json())` — встроенный в Express парсер. Берёт тело запроса в формате JSON и превращает в объект `req.body`. Без этой строки `req.body` был бы `undefined`.
5. `app.use(helmet({ crossOriginResourcePolicy: false }))` — `helmet` ставит защитные заголовки (запрет встраивать сайт в чужие фреймы и т.п.). Одна политика выключена, чтобы фронт с другого адреса мог загружать картинки из `/uploads`.
6. Лимитеры `rateLimit({...})` из `express-rate-limit`:
   - все `/api/` — 100 запросов за 15 минут с одного IP;
   - вход, регистрация, сброс пароля — 10 неудачных попыток за 15 минут (`skipSuccessfulRequests: true` — удачные не считаются). Это защита от подбора пароля;
   - запрос ссылки сброса — 5 в час.
7. `app.use('/uploads', express.static(path.join(__dirname, 'uploads')))` — `express.static` отдаёт файлы из папки как есть. Запрос `/uploads/pink-123.jpg` вернёт файл картинки. `__dirname` — встроенная переменная Node: папка, в которой лежит текущий файл. `path.join` склеивает путь правильно для любой ОС.
8. `app.use('/api/products', productsRoute)` и ещё три роутера — всё, что начинается с `/api/products`, отдаётся в `productsRoute`. Внутри роутера адрес считается уже без префикса.

9. Обработчик ошибок с **четырьмя** аргументами `(err, req, res, next)`. Express узнаёт обработчик ошибок именно по четырём аргументам. Сюда попадают ошибки, которые не поймал контроллер, например от multer (файл больше 8 МБ). Он гарантирует, что клиент получит JSON, а не HTML-страницу ошибки.
10. `start()` — сначала `await connectDB(process.env.MONGO_URI)`, и только после успешного подключения `app.listen(...)`. Так сервер не начнёт принимать запросы, пока нет базы.

`require('dotenv').config()` в начале файла читает `.env` и заполняет `process.env`. `process` — встроенный объект Node с информацией о запущенной программе.

3.2 back/db/connect.js

```
const connectDB = (url) => mongoose.connect(url);
```

`mongoose.connect` — метод библиотеки Mongoose. Открывает соединение с MongoDB и возвращает Promise. После этого **все модели** (User, Product и т.д.) автоматически используют это соединение — их не нужно передавать отдельно. Вынесено в отдельный файл, чтобы и `index.js`, и скрипты из `seed/` подключались одинаково.

3.3 Модели (back/models/) — что такое Schema и model

Это самое важное для понимания вопроса «откуда взялся `findOne`».

```
const userSchema = new mongoose.Schema({ name: {...}, email: {...}, ... }, { timestamps: true });
const User = mongoose.model('User', userSchema);
module.exports = User;
```

Шаг 1. **`new mongoose.Schema({...})`** — описание формы документа: какие поля, какого типа, обязательные или нет. Сама по себе схема ничего не умеет, это просто описание.

Шаг 2. **`mongoose.model('User', userSchema)`** — Mongoose создаёт **класс модели**. Имя `'User'` превращается в имя коллекции в базе: Mongoose переводит в нижний регистр и ставит во множественное число → коллекция `users`.

Шаг 3. Откуда методы. Класс, который вернул `mongoose.model`, наследует от базового класса `Model` из Mongoose готовый набор методов. **Мы их не писали — они приходят из библиотеки.** Поэтому у `User`, `Product`, `Cart`, `Order` есть одни и те же методы: `find`, `findOne`, `findById`, `create`, `findByIdAndUpdate` и т.д. Каждый из них работает со своей коллекцией.

Настройки полей в схемах

Настройка	Что значит	Пример
`type: String` / `Number` / `Date` / `[String]`	тип значения. `[String]` — массив строк	`color_palette: { type: [String] }`
`required: true`	без этого поля документ не сохранится — Mongoose бросит ошибку	`email`, `password`
`unique: true`	в базе создаётся уникальный индекс: два пользователя с одной почтой невозможны	`email`, `orderNumber`

`default: ...`	значение, если поле не передали	`role: 'user'`, `stock: null`
`select: false`	поле **не возвращается** в обычных запросах	`password`, поля сброса пароля
`enum: [...]`	разрешены только значения из списка	`category: ['bags','bracelets']`
`min: 0` / `min: 1`	минимальное число	`stock`, `quantity`
`ref: 'Product'`	поле хранит `_id` документа из другой модели. Нужно для `populate`	`cartModel`, `orderModel`
`{ timestamps: true }`	Mongoose сам добавляет и обновляет `createdAt` и `updatedAt`	во всех четырёх моделях
`{ _id: false }`	вложенному объекту не создавать свой `_id`	позиции заказа `orderItemSchema`

`mongoose.Schema.Types.ObjectId` — тип для `_id`. Каждый документ в MongoDB автоматически получает уникальный `_id` вида `65f1a2b3c4d5e6f7a8b9c0d1` (24 символа).

Четыре модели проекта

User (`userModel.js`) — `name`, `email` (уникальный), `password` (скрыт, `select: false`), `avatar`, `role` (по умолчанию `'user'`; бывает `'manager'` и `'admin'`), `resetPasswordToken` и `resetPasswordExpires` (оба скрыты).

Product (`productModel.js`) — `file_name` (путь к картинке), `model_name`, `color_palette`, `details`, `style` (массивы строк), `category` (только `bags` или `bracelets`), `price`, `stock` (`null` — остаток не отслеживается, товар всегда доступен).

Cart (`cartModel.js`) — `user` (ссылка на User) и `items`: массив `{ product: ссылка на Product, quantity }`. В корзине хранятся **не товары, а только их `_id`**.

Order (`orderModel.js`) — `orderNumber`, `user` (может быть `null` для гостя), `customer { name, email, phone }`, `shipping { address, city, country }`, `deliveryMethod` (`delivery` / `pickup`), `items` (копии товаров на момент покупки: имя, цена, количество, параметры кастомной сумки), `subtotal`, `total`, `status` (6 вариантов), `payment { sessionId, paymentIntentId, paidAt }`, `notes`.

Почему в заказе **копия** имени и цены, а не ссылка: если менеджер потом поменяет цену товара, старый заказ должен показывать ту цену, по которой реально купили.

3.4 Методы Mongoose, которые используются в проекте

Все они — методы модели (класса), полученные из библиотеки. Вызываются как `Модель.метод(...)`.

Метод	Что делает	Что возвращает	Где у нас
`find(условие)`	все документы, подходящие под условие. `find({})` — все	массив (может быть пустым)	список товаров, список заказов
`findOne(условие)`	**первый** документ под условие	документ или `null`	вход, регистрация, корзина пользователя

`findById(id)`	документ по `_id`. То же, что `findOne({ _id: id })`	документ или `null`	товар, заказ, `protect`
`create(данные)`	создаёт и сразу сохраняет документ, проверяя схему	созданный документ	регистрация, товар, заказ, корзина
`документ.save()`	сохранить изменения уже загруженного документа	документ	корзина, сброс пароля, заказ
`findByIdAndUpdate(id, изменения, опции)`	найти по `_id` и обновить. `{ new: true }` — вернуть **новую** версию (без этого вернётся старая). `runValidators: true` — проверить правила схемы при обновлении	документ или `null`	товар, статус заказа
`findByIdAndDelete(id)`	найти и удалить	удалённый документ или `null`	удаление товара
`findOneAndUpdate(условие, изменения)`	как предыдущий, но поиск по условию	документ или `null`	`seed/makeManager.js`
`updateOne(условие, изменения)`	обновить один документ, не загружая его	отчёт об изменении	списание остатка
`deleteMany()` / `insertMany(массив)`	удалить все / вставить много	—	`seed/products.js`
`aggregate([...])`	сложная выборка в несколько этапов (группировка, суммы)	массив результатов	статистика продаж
`.select('+поле')`	добавить к запросу скрытое поле	—	вход (`+password`)
`.populate('путь')`	заменить ссылку `_id` на сам документ	—	корзина
`.sort({ поле: -1 })` / `.limit(n)`	сортировка (−1 — по убыванию) и ограничение количества	—	заказы: новые сверху, максимум 200

Важно про `await`: `User.findOne(...)` сам по себе возвращает объект-запрос (Query), а не данные. К нему можно дописать `.select()`, `.populate()`, `.sort()`. Запрос **реально уходит в базу** только когда на нём сделали `await` (или `.then`).

Операторы MongoDB в условиях (знак $)

Оператор	Значение	Пример у нас
`$gt`	больше	`resetPasswordExpires: { $gt: Date.now() }` — срок ещё не истёк
`$gte`	больше или равно	`stock: { $gte: it.quantity }` — хватает остатка
`$in`	значение входит в список	`status: { $in: PAID_STATUSES }`
`$or`	хотя бы одно условие верно	поиск заказа по номеру, имени **или** почте

`$inc`	увеличить число на значение (минус — уменьшить)	`{ $inc: { stock: -2 } }`
`$match`, `$group`, `$sum`, `$unwind`, `$sort`, `$limit`, `$multiply`	этапы и функции `aggregate`	`getStats` (см. 3.10)

3.5 back/controllers/AuthController.js — построчно

Импорты

```
const bcrypt = require('bcryptjs');        // библиотека: хеширование паролей
const jwt = require('jsonwebtoken');       // библиотека: токены
const crypto = require('crypto');          // встроенный модуль Node: случайные байты, SHA-256
const User = require('../models/userModel.js'); // наш файл: модель пользователя
const { sendMail } = require('../utils/email'); // наш файл: отправка писем
```

Что такое хеш и зачем bcrypt

Хеш — это результат односторонней функции: из пароля `qwerty123` легко получить строку вида `$2b$10$N9qo8uLOickgx2ZMRZoMye...`, но из этой строки **невозможно** получить обратно пароль. В базе хранится только хеш. Если базу украдут — паролей там нет.

`bcrypt.hash(password, 10)` — создаёт хеш.

- Число `10` — «стоимость» (cost, salt rounds): bcrypt выполняет 2^{10} = 1024 раунда вычислений. Это делает подбор пароля перебором очень медленным.
- bcrypt сам генерирует **соль** — случайную добавку, уникальную для каждого пароля. Поэтому у двух пользователей с одинаковым паролем хеши разные, и готовые таблицы хешей (rainbow tables) не помогают.
- Результат — одна строка, в которой записано всё: `$2b$` (версия алгоритма) + `10$` (стоимость) + 22 символа соли + 31 символ самого хеша.

`bcrypt.compare(password, user.password)` — проверяет пароль при входе.

1. Берёт из сохранённого хеша `user.password` соль и стоимость (они там записаны).
2. Хеширует введённый пароль **с той же солью и стоимостью**.
3. Сравнивает результат с сохранённым хешем. Совпало → `true`, нет → `false`.

Пароль при этом **никогда не расшифровывается** — расшифровать хеш нельзя, можно только посчитать заново и сравнить.

Что такое JWT

JWT (JSON Web Token) — строка из трёх частей через точку: `xxxxx.yyyyy.zzzzz`.

1. **Заголовок** — алгоритм подписи (у нас по умолчанию HS256).
2. **Данные (payload)** — то, что мы положили: `{ id, name, email, role }` и срок `exp`.
3. **Подпись** — вычисляется из первых двух частей и `JWT_SECRET`.

Первые две части просто закодированы (Base64), их **может прочитать кто угодно** — поэтому пароль туда не кладут. Но **изменить** их нельзя: подпись перестанет совпадать, а пересчитать её без секрета невозможно.

`jwt.sign(данные, секрет, { expiresIn: '1h' })` — создать токен, живёт 1 час. **`jwt.verify(token, секрет)`** — проверить подпись и срок. Если что-то не так — бросает ошибку. Если всё хорошо — возвращает данные из токена.

Вспомогательные функции файла

- `hashToken(raw)` — `crypto.createHash('sha256').update(raw).digest('hex')`: создать хеш SHA-256 → передать данные → получить результат строкой в шестнадцатеричном виде. Используется для токена сброса пароля.
- `sentRecently(user)` — не даёт запросить вторую ссылку сброса раньше чем через 2 минуты.
- `createToken(user)` — обёртка над `jwt.sign`.
- `publicUser(user)` — выбирает только безопасные поля для отправки клиенту. Пароль сюда не попадает никогда.

signup — регистрация

```
const { name, email, password, avatar } = req.body;   // данные формы (express.json() положил их в req.body)
const exists = await User.findOne({ email });         // есть ли уже такая почта
if (exists) return res.status(400).send({ message: 'Email already exists' });
const hashed = await bcrypt.hash(password, 10);       // хешируем пароль
const user = await User.create({ name, email, password: hashed, avatar: avatar || '' });
const token = createToken(user);                      // сразу выдаём токен — пользователь вошёл
res.status(201).json({ token, user: publicUser(user) });
```

Заметь: `role` в `create` не передаётся, поэтому из формы нельзя зарегистрироваться менеджером — всегда ставится `default: 'user'`.

signin — вход (тот самый пример)

```
const { email, password } = req.body;
const user = await User.findOne({ email }).select('+password');
```

Разбор по частям:

- **`User`** — модель из `userModel.js`, импортирована в начале файла через `require`.
- **`.findOne(...)`** — метод, который `User` получил от Mongoose (см. 3.3). Ищет **первый** документ в коллекции `users`, подходящий под условие.
- **`{ email }`** — короткая запись `{ email: email }`. Условие: поле `email` в базе равно введённой почте.
- **`.select('+password')`** — в схеме у `password` стоит `select: false`, поэтому обычный `findOne` вернул бы пользователя **без** пароля. Плюс перед именем поля значит «включи это скрытое поле в ответ». (Минус, `'-avatar'`, наоборот исключил бы поле.) Пароль нужен только здесь — чтобы сравнить. Во всех остальных местах (например в `protect`) он не загружается и случайно не уйдёт клиенту.
- **`await`** — ждём ответа базы. Результат — документ пользователя или `null`.

```
if (!user) return res.status(400).send({ message: 'Invalid credentials' });
const isMatch = await bcrypt.compare(password, user.password);
if (!isMatch) return res.status(400).send({ message: 'Invalid credentials' });
```

Обрати внимание: и «нет такой почты», и «неверный пароль» дают **одинаковое** сообщение. Так злоумышленник не узнает, зарегистрирована ли почта.

```
const token = createToken(user);
res.status(200).json({ token, user: publicUser(user) });
```

forgotPassword — ссылка для сброса пароля

1. Ищет пользователя по почте (почта приводится к нижнему регистру и без пробелов).
2. Если пользователь есть и ссылку не отправляли последние 2 минуты:
   - `crypto.randomBytes(32).toString('hex')` — 32 случайных байта → строка из 64 символов. Это **сырой токен**.
   - В базу пишется **только хеш** токена (`hashToken`) и срок: сейчас + 60 минут.
   - Сырой токен уходит в письме как ссылка `.../reset-password?token=...`.
3. Ответ всегда одинаковый: «если аккаунт существует, ссылка отправлена». Так нельзя проверить, кто зарегистрирован на сайте.

Почему хранится хеш, а не сам токен: если кто-то прочитает базу, он увидит только хеш. Построить по нему рабочую ссылку нельзя.

resetPassword — установка нового пароля

```
const user = await User.findOne({
  resetPasswordToken: hashToken(token),          // хеш токена из ссылки совпадает с сохранённым
  resetPasswordExpires: { $gt: Date.now() }      // и срок ещё не прошёл
});
```

Если нашёлся — новый пароль хешируется, токен и срок обнуляются (ссылка одноразовая), `user.save()`.

getMe — кто я

Просто возвращает `req.user`. Сам пользователь туда положен middleware `protect`, который выполнился раньше (см. 3.6).

3.6 back/middleware/ — функции-посредники

Middleware — функция с тремя аргументами `(req, res, next)`. Express вызывает их по очереди. Каждая либо:

- вызывает `next()` → запрос идёт к следующей функции в цепочке;
- либо сама отвечает `res.status(...).send(...)` → цепочка обрывается, контроллер не вызывается.

authMiddleware.js — protect

```
const authHeader = req.headers.authorization;           // "Bearer eyJhbGciOi..."
if (!authHeader || !authHeader.startsWith('Bearer ')) return res.status(401)...;
const token = authHeader.split(' ')[1];                  // разрезать по пробелу, взять вторую часть
const decoded = jwt.verify(token, process.env.JWT_SECRET); // проверить подпись и срок
const user = await User.findById(decoded.id);            // найти пользователя по id из токена
req.user = { id, name, email, avatar, role };            // положить на запрос
next();                                                  // пропустить дальше
```

- `req.headers` — объект со всеми заголовками запроса. Заголовок `Authorization` фронт добавляет функцией `authHeader` из `api.jsx`.
- «Bearer» — стандартное слово-префикс для токенов («предъявитель»).
- **Роль берётся из базы, а не из токена.** В токене роль тоже есть, но если менеджера разжаловали, его старый токен ещё действует. Поэтому `protect` каждый раз спрашивает базу.
- `req.user` — мы сами добавляем это поле к объекту запроса. Все следующие функции в цепочке его видят.

roleMiddleware.js — restrictTo

```
const restrictTo = (...roles) => {
  return (req, res, next) => {
    if (!req.user || !roles.includes(req.user.role)) return res.status(403)...;
    next();
  };
};
```

Это **функция, которая возвращает функцию**. `restrictTo('manager', 'admin')` вызывается один раз при запуске и создаёт middleware, который «помнит» список ролей (это называется **замыкание**, closure). `...roles` (rest-параметр) собирает все переданные аргументы в массив `['manager', 'admin']`. `roles.includes(x)` — есть ли `x` в массиве.

Порядок важен: `restrictTo` стоит **после** `protect`, потому что ему нужен `req.user`.

401 против 403: 401 — «я не знаю, кто ты» (нет токена). 403 — «я знаю, кто ты, но тебе нельзя».

upload.js — загрузка картинок (multer)

`multer` — библиотека, которая умеет читать формы с файлами (`multipart/form-data`). Обычный `express.json()` файлы не понимает.

- `multer.diskStorage({ destination, filename })` — сохранять на диск. Обе функции получают `cb` (callback) — функцию, которую нужно вызвать с результатом: `cb(null, значение)`. Первый аргумент `null` значит «ошибки нет».
- `destination` — папка `back/uploads`. `fs.mkdirSync(dir, { recursive: true })` создаёт её, если нет.
- `filename` — имя файла: исходное имя, очищенное от странных символов (`replace(/[^a-z0-9]/gi, '-')` — всё, кроме букв и цифр, заменить на дефис), обрезанное до 40 символов, плюс `Date.now()` (текущее время в миллисекундах), чтобы два файла с одинаковым именем не затёрли друг друга.
- `fileFilter` — пропускает только файлы с типом `image/...`. Иначе — ошибка.
- `limits: { fileSize: 8 * 1024 * 1024 }` — не больше 8 МБ.

В роутере используется как `upload.single('image')`: «в форме одно поле-файл с именем `image`». После этого файл доступен в контроллере как `req.file`, а текстовые поля формы — как обычно в `req.body`.

validate.js — проверка форм

`validateSignup` и `validateSignin` проверяют `req.body` до контроллера: что поля — строки, длина имени 2–60, почта по регулярному выражению, пароль 8–128 символов. Если что-то не так — `400` и контроллер не вызывается. Если всё хорошо — очищенные значения (без пробелов, почта в нижнем регистре) записываются обратно в `req.body` и `next()`.

Регулярное выражение почты `/^[^\s@]+@[^\s@]+\.[^\s@]+$/` означает: «что-то без пробелов и @, потом @, потом что-то, точка, что-то».

Зачем проверять на сервере, если на фронте есть `required`: проверки в браузере легко обойти — можно отправить запрос напрямую, минуя сайт. Сервер обязан проверять сам.

3.7 back/routes/ — какой адрес куда ведёт

```
const router = express.Router();          // мини-приложение для группы адресов
router.post('/signin', validateSignin, signin);
module.exports = router;
```

`express.Router()` — функция Express, создаёт отдельный набор маршрутов. В `index.js` он подключается с префиксом: `app.use('/api/auth', authRoute)` → полный адрес `/api/auth/signin`.

Аргументы `router.метод(адрес, функция1, функция2, ..., контроллер)` выполняются **слева направо**. Всё, что между адресом и контроллером, — middleware.

`:id` в адресе — параметр. Запрос `/api/products/65f1a...` → в контроллере `req.params.id === '65f1a...'`.

Полная таблица API

Метод и адрес	Цепочка	Кто может
`POST /api/auth/signup`	`validateSignup` → `signup`	все
`POST /api/auth/signin`	`validateSignin` → `signin`	все
`POST /api/auth/forgot-password`	`forgotPassword`	все
`POST /api/auth/reset-password`	`resetPassword`	все
`GET /api/auth/me`	`protect` → `getMe`	вошедшие
`GET /api/products`	`getAllProducts`	все
`GET /api/products/:id`	`getProductById`	все
`POST /api/products`	`protect` → `restrictTo` → `upload.single('image')` → `createProduct`	менеджер/ админ
`PUT /api/products/:id`	`protect` → `restrictTo` → `upload.single('image')` → `updateProduct`	менеджер/ админ

`DELETE /api/products/:id`	`protect` → `restrictTo` → `deleteProduct`	менеджер/ админ
`GET /api/cart`	`protect` → `getCart`	вошедшие
`POST /api/cart`	`protect` → `addToCart`	вошедшие
`PATCH /api/cart/:productId`	`protect` → `updateQuantity`	вошедшие
`DELETE /api/cart/:productId`	`protect` → `removeFromCart`	вошедшие
`DELETE /api/cart`	`protect` → `clearCart`	вошедшие
`POST /api/orders/checkout-session`	`createCheckoutSession`	все (гость тоже)
`GET /api/orders/by-session/:sessionId`	`getOrderBySession`	все
`POST /api/orders/webhook`	`express.raw` → `stripeWebhook` (в `index.js`)	Stripe
`GET /api/orders`	`protect` → `restrictTo` → `getAllOrders`	менеджер/ админ
`GET /api/orders/stats`	`protect` → `restrictTo` → `getStats`	менеджер/ админ
`PATCH /api/orders/:id/status`	`protect` → `restrictTo` → `updateOrderStatus`	менеджер/ админ
`GET /api/orders/:id`	`protect` → `restrictTo` → `getOrderById`	менеджер/ админ

В `ordersRoute.js` защита собрана в массив `const manager = [protect, restrictTo('manager', 'admin')]` — Express понимает массив middleware так же, как перечисление через запятую.

Почему `/stats` объявлен **до** `/:id`: Express проверяет маршруты сверху вниз. Если бы `/:id` стоял первым, адрес `/stats` подошёл бы под него с `id = 'stats'`.

3.8 ProductController.js — товары

- `toArray(value)` — форма отправляет цвета строкой `"Pink, White"`. `split(',')` режет по запятой, `map(item => item.trim())` убирает пробелы, `filter(Boolean)` выбрасывает пустые строки. Получаем `['Pink', 'White']`.
- `parseStock(value)` — пустое поле → `null` (остаток не отслеживается), иначе целое число не меньше 0.
- `buildFilePath(file)` — `uploads/имя-файла.jpg`. `replace(/\\/g, '/')` меняет обратные слеши Windows на обычные.
- `removeImageFile(fileName)` — `fs.promises.unlink` удаляет файл с диска. `.catch(() => {})` — если файла уже нет, ошибку игнорируем.

Функция	Что делает
`createProduct`	проверяет, что есть картинка (`req.file`) и обязательные поля; `Product.create(...)`. При ошибке удаляет уже загруженную картинку, чтобы не копился мусор

`getAllProducts`	`Product.find({})` — все товары. Ответ `{ count, data }`
`getProductById`	`Product.findById(req.params.id)` ; нет → 404
`updateProduct`	собирает объект `update` только из присланных полей; если пришла новая картинка — запоминает старую и удаляет её после успешного обновления
`deleteProduct`	`Product.findByIdAndDelete` , затем удаляет файл картинки

3.9 CartController.js — корзина вошедшего пользователя

- `loadCart(cartId)` — `Cart.findById(cartId).populate('items.product')` .

Что делает `populate` : в базе корзина выглядит так: `items: [{ product: '65f1a...', quantity: 2 }]` — только id. `populate('items.product')` говорит Mongoose: «по полю `ref: 'Product'` сходи в коллекцию товаров и подставь вместо id весь документ товара». После этого `item.product.model_name` , `item.product.price` доступны. Это похоже на JOIN в SQL.

Затем `filter((item) => item.product)` убирает позиции, чей товар удалили из каталога (`populate` вернул для них `null`).

Функция	Что делает
`getCart`	`Cart.findOne({ user: req.user.id })` ; если корзины нет — создаёт пустую
`addToCart`	`findIndex` ищет товар в корзине. `item.product.toString() === productId` — id в базе хранится как объект ObjectId, его нужно превратить в строку для сравнения. Есть → `quantity += 1` , нет → `push` . Затем `cart.save()`
`removeFromCart`	оставляет все позиции, кроме удаляемой (`filter`), `cart.save()`
`updateQuantity`	`parseInt` превращает присланное количество в целое число; меньше 1 → 400. `find` находит позицию, меняет `quantity` , `cart.save()`
`clearCart`	`Cart.findOneAndUpdate({ user }, { items: [] })` — очистить корзину одним запросом (после оплаты или по кнопке «очистить»)

`req.user.id` берётся из `protect` — пользователь может работать только со **своей** корзиной, чужой id подставить нельзя.

3.10 OrderController.js — заказы и оплата

Подключение Stripe

```
const stripe = process.env.STRIPE_SECRET_KEY ? require('stripe')(process.env.STRIPE_SECRET_KEY) :
null;
```

Тернарный оператор `условие ? да : нет` . Если ключа нет — `stripe = null` , и оформление отвечает `501` . `require('stripe')` возвращает функцию, её вызов с ключом даёт объект для работы с API Stripe.

Вспомогательные функции

- `genOrderNumber()` — номер вида `DH-20261021-4821`: дата + случайные 4 цифры. `padStart(2, '0')` превращает `5` в `05`.
- `userIdFromHeader(req)` — если покупатель вошёл, достаёт его id из токена. Если нет или токен плохой — `null` (гость). Здесь не используется `protect`, потому что гость тоже может купить.
- `buildOrderItems(rawItems)` — **ключевая функция безопасности.** Для каждой позиции из корзины:
  - `Math.max(1, parseInt(it.quantity, 10) || 1)` — количество, не меньше 1.
  - `mongoose.Types.ObjectId.isValid(it.id)` — проверяет, похож ли id на настоящий id MongoDB.
  - Если товар найден в базе — **цена, имя и картинка берутся из базы**, а не из запроса. Проверяется остаток.
  - Если не найден — это кастомная сумка из конструктора. Её цена **тоже считается на сервере**: по таблицам `SIZE_PRICES` и `STRAP_PRICES` в начале файла (размер + ремешок). Цена из запроса не используется совсем. Если размер или ремешок неизвестны — ошибка, заказ не создаётся.
  - Почему `typeof sizePrice !== 'number'`, а не просто `!sizePrice`: у любого объекта JavaScript есть встроенные свойства (например `constructor`). Если прислать `size: 'constructor'`, `SIZE_PRICES[...]` вернёт не число, а функцию. Проверка типа отсекает такие подмены.

createCheckoutSession — начало оплаты

1. Проверка: есть имя, почта, корзина не пустая, Stripe настроен.
2. `buildOrderItems` → позиции и, возможно, ошибка остатка.
3. `items.reduce((sum, it) => sum + it.price * it.quantity, 0)` — `reduce` проходит по массиву и накапливает сумму, начиная с 0.
4. `Order.create({...status: 'pending'})` — заказ создаётся **до** оплаты со статусом «ожидает».
5. `stripe.checkout.sessions.create({...})` — запрос в Stripe: создать страницу оплаты.
   - `line_items` — позиции. `unit_amount: Math.round(it.price * 100)` — Stripe принимает сумму в **центах**.
   - `metadata: { orderId }` — наша пометка: какой заказ оплачивается. Stripe вернёт её в вебхуке.
   - `success_url` / `cancel_url` — куда вернуть покупателя. `{CHECKOUT_SESSION_ID}` Stripe сам заменит на id сессии.
6. id сессии сохраняется в заказе, клиенту отправляется `{ url }` — адрес страницы оплаты Stripe.

stripeWebhook — Stripe сообщает «оплачено»

Вебхук — это когда **внешний сервис сам отправляет запрос нашему серверу**. После оплаты Stripe делает `POST /api/orders/webhook`.

1. `stripe.webhooks.constructEvent(req.body, подпись, STRIPE_WEBHOOK_SECRET)` — проверяет, что запрос действительно от Stripe (подпись в заголовке `stripe-signature`). Подделать нельзя без секрета. Для этого и нужно сырое тело (`express.raw`).
2. Если событие `checkout.session.completed` — находим заказ по `session.metadata.orderId`.

3. Если он ещё `pending`: статус `paid`, сохраняем id платежа и время.
4. **Списание остатка:**

```
Product.updateOne({ _id: it.product, stock: { $gte: it.quantity } }, { $inc: { stock: -
it.quantity } })
```

Условие «остаток ≥ количества» — часть самого запроса. База проверяет и уменьшает **за одну операцию** (атомарно), поэтому остаток не уйдёт в минус, даже если два покупателя платят одновременно.

5. Письма покупателю (`sendCustomerReceipt`) и менеджеру (`sendManagerReport`). Ошибка письма не ломает обработку заказа — она в отдельном `try/catch`.
6. Ответ `{ received: true }` — Stripe должен получить 200, иначе будет повторять запрос.

Проверка `order.status === 'pending'` защищает от повторной обработки, если Stripe пришлёт одно событие дважды.

Почему оплату подтверждает вебхук, а не страница «спасибо» в браузере: покупатель может закрыть вкладку сразу после оплаты, и браузер ничего не сообщит. А ещё запрос из браузера можно подделать. Вебхук приходит от Stripe напрямую и подписан.

Остальные функции

Функция	Что делает
`getOrderBySession`	для страницы «спасибо»: по id сессии Stripe возвращает номер заказа, статус, сумму и почту (не весь заказ)
`getAllOrders`	список для менеджера. Фильтр по статусу; поиск: `new RegExp(search, 'i')` — регулярное выражение без учёта регистра, `$or` по номеру, имени и почте. `.sort({ createdAt: -1 })` — новые сверху, `.limit(200)`
`getOrderById`	один заказ
`updateOrderStatus`	проверяет, что статус из списка `ALLOWED_STATUS`, затем `findByIdAndUpdate`
`getStats`	статистика через `aggregate` (ниже)

aggregate — как считается статистика

`aggregate` принимает массив **этапов**. Данные проходят через них по очереди, как по конвейеру.

```
Order.aggregate([
  { $match: match },                // 1. оставить только оплаченные заказы за период
  { $group: {                       // 2. сгруппировать ВСЕ в одну группу (_id: null)
      _id: null,
      revenue: { $sum: '$total' },  //    сложить поле total
      count: { $sum: 1 }            //    посчитать количество (по 1 за каждый)
  } }
]);
```

Знак `$` перед именем поля (`'$total'`) значит «значение этого поля документа».

Топ товаров:

- `$unwind: '$items'` — «развернуть» массив: заказ с 3 позициями превращается в 3 отдельных документа, по одному на позицию.
- `$group` по `$items.name` — сложить количество и выручку (`$multiply` — цена × количество).
- `$sort: { qty: -1 }` — по убыванию, `$limit: 5` — первые пять.

`const [totals] = await Order.aggregate(...)` — `aggregate` всегда возвращает массив; квадратные скобки берут первый элемент.

3.11 back/utils/email.js — письма

```
const { Resend } = require('resend');
const resend = process.env.RESEND_API_KEY ? new Resend(process.env.RESEND_API_KEY) : null;
```

`Resend` — класс из библиотеки, `new` создаёт объект для отправки. `sendMail({ to, subject, html })` вызывает `resend.emails.send(...)`. Если ключа нет — письмо не отправляется, а печатается в консоль: сайт продолжает работать при разработке.

Особенность: библиотека Resend **не бросает ошибку**, а возвращает `{ data, error }`. Поэтому ошибку проверяем вручную и бросаем сами (`throw new Error(...)`).

Почему Resend, а не Gmail: отправка через почтовый ящик (SMTP) требует открытого порта на хостинге и своего домена; Resend работает через обычный HTTPS-запрос.

3.12 back/seed/ — служебные скрипты

Запускаются вручную из терминала, сайт их не вызывает.

- `products.js` — `node seed/products.js`: подключается к базе, `Product.deleteMany()` удаляет все товары, `Product.insertMany(products)` вставляет товары из `productsData.json`. `require` умеет сразу читать JSON-файлы.
- `makeManager.js` — `node seed/makeManager.js почта@x.com manager`: назначает роль. `process.argv` — массив слов из командной строки: `[0]` — путь к node, `[1]` — путь к скрипту, `[2]` — почта, `[3]` — роль. `process.exit(0)` — завершить программу (0 — успех, 1 — ошибка).

Почему роль назначается скриптом, а не на сайте: так нет ни одного адреса API, через который можно стать менеджером.

Часть 4. Фронтенд (front/) по файлам

4.1 Как запускается сайт

1. `front/index.html` содержит пустой `<div id="root"></div>` и подключает `src/main.jsx`.
2. **Vite** (`npm run dev`) запускает сервер разработки на `localhost:5173`, на лету превращает JSX в обычный JavaScript, а `@tailwindcss/vite` — классы Tailwind в CSS. Настройка — `vite.config.js`: два плагина, `react()` и `tailwindcss()`.
3. `main.jsx` рисует приложение внутрь `#root`.

JSX — HTML-подобная запись внутри JavaScript: `<h1>{title}</h1>`. В фигурных скобках — любое JS-выражение. Браузер JSX не понимает, его переводит Vite. Атрибут `class` в JSX пишется как `className`.

4.2 main.jsx

```
createRoot(document.getElementById('root')).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

- `createRoot` (из `react-dom/client`) — подключить React к HTML-элементу.
- `BrowserRouter` (из `react-router-dom`) — включает навигацию: следит за адресом в браузере. Оборачивает **всё** приложение, чтобы `Link`, `useNavigate` и т.д. работали в любом компоненте.
- `import './index.css'` — подключение стилей (в том числе Tailwind).

4.3 Хуки React — что это и откуда

Хук — функция React, имя начинается с `use`. Вызывается только внутри компонента, на верхнем уровне (не внутри `if` или цикла).

Хук	Откуда	Что делает	Пример у нас
`useState(начальное)`	`react`	хранит значение компонента. Возвращает `[значение, функция-изменения]`. При вызове функции-изменения компонент перерисовывается	`const [email, setEmail] = useState("")`
`useEffect(функция, [зависимости])`	`react`	выполнить код **после** отрисовки. `[]` — один раз при открытии. `[token]` — заново при каждом изменении `token`. Функция, которую вернули изнутри, вызывается при уходе со страницы (очистка)	загрузка товаров, `/api/auth/me`
`useRef(начальное)`	`react`	хранит значение между отрисовками, но его изменение **не** перерисовывает компонент	`prevToken` в корзине

`createContext` / `useContext`	`react`	общие данные для всех компонентов без передачи через пропсы	корзина
`useNavigate()`	`react-router-dom`	функция перехода на другую страницу из кода: `navigate('/admin')`	после входа
`useParams()`	`react-router-dom`	параметры из адреса: `/products/:id` → `{ id }`	страница товара
`useSearchParams()`	`react-router-dom`	параметры после `?`: `?token=abc` → `params.get('token')`	сброс пароля, «спасибо»

Пропсы (props) — данные, которые родитель передаёт компоненту, как атрибуты: `<ManagerProductsPage token={token} />` → внутри `({ token }) => ...`.

4.4 App.jsx — главный компонент и маршруты

Состояние пользователя

```
const [token, setToken] = useState(localStorage.getItem("token"));
const [user, setUser] = useState(readUser);
```

`localStorage` — хранилище браузера (встроенное, не библиотека). Хранит строки, переживает перезагрузку страницы. `getItem` / `setItem` / `removeItem`. Объект перед сохранением превращают в строку `JSON.stringify`, при чтении обратно — `JSON.parse`. `readUser` обёрнут в `try/catch`, потому что битая строка сломала бы `JSON.parse`.

`useState(readUser)` — передана **функция**, а не её результат: React вызовет её только один раз при первой отрисовке.

Два эффекта

1. Подписка на событие `storage` — если пользователь вышел в другой вкладке, эта вкладка тоже узнает. `return () => window.removeEventListener(...)` — отписаться при закрытии.
2. При каждом изменении `token` — запрос `GET /api/auth/me` с токеном. Сервер возвращает актуального пользователя с ролью **из базы**. Так роль нельзя подделать, отредактировав `localStorage` в инструментах разработчика (а даже если подделать — сервер всё равно не пустит, у него своя проверка в `protect`).

Маршруты

```
<Routes>
  <Route path="/" element={<HomePage />} />
  <Route path="/products/:id" element={<ProductDetailPage />} />
  <Route path="/admin" element={isManager ? <ManagerProductsPage token={token} /> : <Navigate
to="/login" replace />} />
</Routes>
```

- `Routes` смотрит на текущий адрес и рисует **один** подходящий `Route`.

- `:id` — параметр, читается через `useParams`.
- `Navigate` — немедленный переход. `replace` — заменить запись в истории, чтобы кнопка «назад» не вернула на запрещённую страницу.
- `isManager` — роль `manager` или `admin`.

Защита на фронте — только для удобства. Настоящая защита на сервере (`protect` + `restrictTo`). Если кто-то откроет страницу менеджера в обход, сервер всё равно ответит 401/403 на каждый запрос.

`Header` и `Footer` стоят вне `Routes` — они видны на всех страницах. Всё обёрнуто в `CartProvider` — корзина доступна везде.

4.5 api.jsx — связь с сервером

```
const api = axios.create({ baseURL: API_BASE_URL });
```

`axios` — библиотека для HTTP-запросов. `axios.create` создаёт настроенную копию: теперь `api.get('/api/products')` автоматически пойдёт на `http://localhost:5000/api/products`. Адрес сервера записан **в одном месте**.

- `API_BASE_URL` — если сайт открыт на `localhost`, сервер локальный; иначе — `VITE_API_URL` или адрес на Render.
- `imageUrl('uploads/pink.jpg')` → `http://localhost:5000/uploads/pink.jpg`. `replace(/^\//, '')` убирает лишний слеш в начале.
- `authHeader(token)` → `{ headers: { Authorization: 'Bearer ...' } }` — третий аргумент для `api.post` / второй для `api.get`. Именно этот заголовок читает `protect` на сервере.

Методы axios: `api.get(url, config)`, `api.post(url, body, config)`, `api.put(...)`, `api.patch(...)`, `api.delete(url, config)`. Ответ сервера лежит в `response.data` (то, что сервер отправил через `res.json(...)`). Поэтому список товаров — это `response.data.data`: первое `data` — от axios, второе — поле, которое мы сами назвали на сервере.

Если сервер ответил кодом 4xx/5xx, axios вызывает `.catch`, а сообщение сервера лежит в `error.response.data.message`.

4.6 context/CartContext.jsx — корзина

Зачем контекст. Корзина нужна шапке (счётчик), каталогам (кнопка «в корзину»), странице корзины и оформлению. Без контекста её пришлось бы передавать пропсами через каждый промежуточный компонент.

```
const CartContext = createContext();               // создать «канал»
<CartContext.Provider value={{ items, addToCart, ... }}>{children}</CartContext.Provider>  // раздать
const useCart = () => useContext(CartContext);     // получить в любом компоненте
```

`children` — всё, что вложено внутрь `<CartProvider>...</CartProvider>` в `App.jsx`, то есть всё приложение.

Две ветки: гость и вошедший

	Гость (нет токена)	Вошедший
Где хранится	`localStorage`, ключ `guestCart`	товары каталога — на сервере, коллекция `carts`; кастомные сумки — `localStorage`, ключ `customItems`
Добавить	меняем массив, сохраняем в `localStorage`	`POST /api/cart`, потом `fetchCart()`
Изменить количество	меняем массив, сохраняем	`PATCH /api/cart/:id`
Удалить	`filter` и сохраняем	`DELETE /api/cart/:id`, потом `fetchCart()`
Очистить	`localStorage.removeItem`	`DELETE /api/cart`

Почему кастомные сумки отдельно. Серверная корзина хранит только ссылки на товары из каталога (`ref: 'Product'`). У сумки из конструктора нет записи в базе, поэтому у вошедшего пользователя она хранится в браузере под ключом `customItems`. `fetchCart` склеивает два списка: `[...товары с сервера, ...loadCustomItems()]`. Отличить кастомную сумку легко: её id начинается с `custom-` (функция `isCustom`). При выходе из аккаунта `Header` стирает `customItems`, чтобы сумки не достались следующему человеку за этим компьютером.

Изменение количества. Новое число сразу показывается на экране (`setItems`), а запрос `PATCH` сохраняет его на сервере. Если запрос не удался — `fetchCart()` вернёт настоящее значение с сервера.

- `toCartItem(item)` — превращает строку корзины с сервера (с вложенным `product` после `populate`) в простой объект для страниц `{ id, name, image, price, quantity }`.
- **Перенос корзины при входе.** `useRef` хранит предыдущий токен. Если раньше токена не было, а теперь есть (`justLoggedIn`), гостевая корзина делится на две части: кастомные сумки переезжают в `customItems`, товары каталога отправляются на сервер — по одному запросу на каждую единицу (`flatMap` + `Array.from({ length: quantity })`). `Promise.allSettled` ждёт **все** запросы, даже если какие-то упали, затем гостевая корзина стирается и загружается серверная.
- `addCustomItem` — сумка из конструктора. Гость — в `guestCart`, вошедший — в `customItems`.
- `getTotalPrice` / `getTotalItems` — `reduce` по позициям.

4.7 Страницы (front/src/pages/)

LoginPage.jsx — вход и регистрация

- Одна форма на два режима, `isLogin` переключает адрес: `/api/auth/signin` или `/api/auth/signup`.
- `e.preventDefault()` — без этого браузер перезагрузил бы страницу при отправке формы.
- `applyAuth` — сохраняет токен и пользователя в `localStorage`, обновляет состояние в `App` (функции `setToken`, `setUser` пришли пропсами), переводит менеджера на `/admin`, остальных — на главную.
- Ошибка сервера («Invalid credentials») показывается под формой.

Header.jsx — шапка

- `NavLink` — как `Link`, но знает, активна ли ссылка. `end` у главной — активна только на точном `/`.
- Ссылка **MANAGE** добавляется в список только менеджеру: `...(isManager ? [{...}] : [])`.
- `handleLogout` — стирает токен и пользователя из `localStorage` и состояния, переходит на главную. На сервере выходить не нужно: JWT сервер не хранит, токен просто перестаёт отправляться.
- Иконки `ShoppingBag`, `User`, `LogOut`, `Menu`, `X` — компоненты из `lucide-react`.
- Мобильное меню — `menuOpen` в `useState`.

BeadedBagsPage.jsx и BraceletsPage.jsx — каталоги

`useEffect(..., [])` при открытии: `GET /api/products`, затем `filter(p => p.category === 'bags')` на стороне браузера. `items.some(item => item.id === p._id)` — уже в корзине или нет (`some` — есть ли хоть один подходящий элемент).

ProductDetailPage.jsx — карточка товара

`const { id } = useParams()` → `GET /api/products/${id}`. Зависимость `[id]` — если перейти на другой товар, запрос повторится. Три состояния: загрузка, «не найден», товар. `product.stock === 0` — «нет в наличии».

CustomStudioPage.jsx — конструктор сумки

- Константы `SIZES`, `STRAPS`, `COLORS` — варианты с ценами и картинками. Картинки подключены через `import` — Vite сам подставит правильный адрес файла.
- Шаги: Size → Strap → Color (`step` в `useState`).
- `config` — выбранные параметры. `updateConfig(patch)` — `setConfig((c) => ({ ...c, ...patch }))`: функция получает предыдущее значение и возвращает новое.
- Цена: `sizeInfo.price + strapInfo.price`. `find` находит выбранный вариант в массиве.
- `handleAddToCart` — `addCustomItem` с `id: 'custom-' + Date.now()` и объектом `config` (размер, ремешок, цвет), затем через `useEffect` с `setTimeout` переход в корзину через 1,2 секунды. `clearTimeout` в функции очистки — если пользователь ушёл раньше, переход не случится.

CartPage.jsx — корзина

Берёт из `useCart()` позиции и функции. Кнопки плюс/минус — `updateQuantity`, мусорка — `removeFromCart`. Пустая корзина — отдельный экран.

CheckoutPage.jsx — оформление

1. Форма в одном объекте `form`. `handleChange` обновляет поле по атрибуту `name` инпута.
2. `validate()` — проверка в браузере (для удобства, сервер проверяет сам). Адрес обязателен только при доставке.
3. Токен отправляется, только если есть — гость тоже может купить.
4. `POST /api/orders/checkout-session` → ответ `{ url }` → `window.location.href = url` — полный переход браузера на страницу оплаты Stripe (это внешний сайт, поэтому не `navigate`).

OrderSuccessPage.jsx — «спасибо»

Stripe возвращает сюда с `?session_id=...`. `useSearchParams` читает его, `clearCart()` очищает корзину, `GET /api/orders/by-session/:id` показывает номер заказа.

ForgotPasswordPage.jsx и ResetPasswordPage.jsx

- Forgot: почта → `POST /api/auth/forgot-password` → показывает одинаковое сообщение.
- Reset: токен из адреса (`?token=...` через `useSearchParams`), проверка длины и совпадения двух паролей, `POST /api/auth/reset-password`, через 1,8 с переход на вход. Нет токена в адресе — «Invalid link».

ManagerProductsPage.jsx — товары менеджера (`/admin`)

- `loadProducts()` — `GET /api/products`.
- Одна форма для создания и редактирования: если `editingId` есть — `PUT /api/products/:id`, иначе `POST /api/products`.
- **`FormData`** — встроенный объект браузера для форм с файлами. `fd.append('image', imageFile)` — тот самый файл, который на сервере прочитает `upload.single('image')`. Axios, получив `FormData`, сам ставит тип `multipart/form-data`.
- `window.confirm(...)` — стандартное окно «OK / Отмена» перед удалением.
- Фильтр по категории — `filter` в браузере.

ManagerOrdersPage.jsx — заказы менеджера (`/admin/orders`)

- `URLSearchParams` — встроенный объект для сборки `?status=paid&search=anna`.
- Эффект с зависимостью `[filter]` — перезагрузка при смене фильтра; с `[period]` — статистика за 7/30 дней или всё время.
- `changeStatus` — `PATCH /api/orders/:id/status`, затем `setOrders(prev => prev.map(...))` заменяет в списке только изменённый заказ, без повторной загрузки.
- `StatCard` — маленький компонент внутри файла для карточки статистики.

4.8 Tailwind — как читать классы

```
className="bg-black text-white py-12 px-6 md:px-8 text-[18px] tracking-[3px]"
```

- `bg-black` — чёрный фон, `text-white` — белый текст;
- `py-12` — отступ сверху и снизу, `px-6` — слева и справа;
- `md:px-8` — на экранах от «среднего» размера отступ больше (адаптивность);
- `text-[18px]` — квадратные скобки: точное значение, 18 пикселей;
- `hover:opacity-70` — при наведении.

Tailwind на сборке оставляет в CSS только те классы, которые реально встречаются в коде.

Часть 5. Полные цепочки «кто кого вызывает»

5.1 Вход в аккаунт

```
LoginPage.jsx  handleSubmit
   │  api.post('/api/auth/signin', { email, password })          (api.jsx → axios)
   ▼
index.js       authLimiter (не больше 10 неудачных за 15 минут)
   │  app.use('/api/auth', authRoute)
   ▼
authRoute.js   router.post('/signin', validateSignin, signin)
   ▼
validate.js    validateSignin → строки? не пустые? почта в нижний регистр → next()
   ▼
AuthController.js  signin
   │  User.findOne({ email }).select('+password')   ──▶ MongoDB, коллекция users
   │  bcrypt.compare(password, user.password)
   │  jwt.sign({ id, name, email, role }, JWT_SECRET, { expiresIn: '1h' })
   │  res.json({ token, user })
   ▼
LoginPage.jsx  applyAuth
   │  localStorage.setItem('token' / 'user')
   │  setToken(...)  → App.jsx перерисовывается
   │  navigate('/admin' или '/')
   ▼
App.jsx        useEffect([token]) → GET /api/auth/me → protect → getMe → роль из базы
CartContext    useEffect([token]) → перенос гостевой корзины → GET /api/cart
```

5.2 Менеджер добавляет товар с картинкой

```
ManagerProductsPage.jsx handleSubmit
   │  new FormData() + append(поля) + append('image', файл)
   │  api.post('/api/products', fd, authHeader(token))
   ▼
productsRoute.js  router.post('/', protect, restrictTo('manager','admin'), upload.single('image'),
createProduct)
   ▼  protect       — токен верный? пользователь есть? → req.user
   ▼  restrictTo    — роль manager или admin? иначе 403
   ▼  upload.single — картинка? до 8 МБ? → файл в back/uploads/, данные в req.file
   ▼  createProduct — Product.create({ file_name: 'uploads/...', ... }) ──▶ MongoDB
   ▼
ManagerProductsPage.jsx  .then → сообщение «Product added», resetForm(), loadProducts()
```

Картинка потом показывается через `imageUrl(product.file_name)` → `http://сервер/uploads/имя.jpg` → `express.static` в `index.js` отдаёт файл.

5.3 Покупка и оплата

```
CheckoutPage.jsx   POST /api/orders/checkout-session  { items, customer, shipping, ... }
   ▼
OrderController.createCheckoutSession
   │  buildOrderItems — цены и остатки из базы
   │  Order.create({ status: 'pending' })                ──▶ MongoDB orders
   │  stripe.checkout.sessions.create({ metadata: { orderId } }) ──▶ Stripe
   │  res.json({ url })
   ▼
CheckoutPage.jsx   window.location.href = url   → страница оплаты Stripe
   ▼  (покупатель вводит карту)
Stripe ──▶ POST /api/orders/webhook   (index.js, express.raw)
   ▼
OrderController.stripeWebhook
   │  constructEvent — проверка подписи
   │  заказ → 'paid', сохранить paymentIntentId, paidAt
   │  Product.updateOne(... $inc: { stock: -qty })       ──▶ MongoDB products
   │  sendCustomerReceipt, sendManagerReport             ──▶ Resend
   ▼
Stripe перенаправляет браузер → /order-success?session_id=...
   ▼
OrderSuccessPage.jsx  clearCart(); GET /api/orders/by-session/:id → номер заказа
```

5.4 Карта импортов (кто кого подключает)

Сервер:

```
index.js
 ├─ db/connect.js ──────────────── mongoose
 ├─ routes/authRoute.js
 │   ├─ controllers/AuthController.js ── bcryptjs, jsonwebtoken, crypto, models/userModel,
utils/email
 │   ├─ middleware/validate.js
 │   └─ middleware/authMiddleware.js ─── jsonwebtoken, models/userModel
 ├─ routes/productsRoute.js
 │   ├─ controllers/ProductController.js ── path, fs, models/productModel
 │   ├─ middleware/authMiddleware.js
 │   ├─ middleware/roleMiddleware.js
 │   └─ middleware/upload.js ──────────── multer, path, fs
 ├─ routes/cartRoute.js
 │   ├─ controllers/CartController.js ─── models/cartModel
 │   └─ middleware/authMiddleware.js
 ├─ routes/ordersRoute.js
 │   ├─ controllers/OrderController.js ── mongoose, jsonwebtoken, stripe, models/orderModel,
models/productModel, utils/email
 │   ├─ middleware/authMiddleware.js
 │   └─ middleware/roleMiddleware.js
 └─ controllers/OrderController.js (stripeWebhook)
utils/email.js ── resend
```

Фронт:

```
main.jsx ── react-dom/client, react-router-dom (BrowserRouter), App.jsx, index.css
App.jsx
 ├─ api.jsx ── axios
 ├─ context/CartContext.jsx ── api.jsx
 ├─ shared/ui/Header.jsx ── CartContext (useCart), lucide-react
 ├─ shared/ui/Footer.jsx
 └─ pages/* ── большинство используют api.jsx и/или CartContext (useCart)
```

Часть 6. Вопросы на защите и ответы

Откуда у модели `User` метод `findOne`, если мы его не писали? `User` создан вызовом `mongoose.model('User', userSchema)`. Mongoose возвращает класс, который наследует от своего базового класса `Model` готовые методы: `find`, `findOne`, `findById`, `create` и другие. Мы их только вызываем.

Что делает `.select('+password')`? В схеме у пароля `select: false` — он не загружается в обычных запросах. Плюс в `select` принудительно добавляет скрытое поле. Нужно только при входе, чтобы сравнить пароль.

Как работает `bcrypt.compare`, если хеш нельзя расшифровать? Не расшифровывает. Из сохранённого хеша берёт соль и стоимость, хеширует ими введённый пароль и сравнивает два хеша.

Что такое соль? Случайная добавка к паролю перед хешированием, своя для каждого пользователя. Одинаковые пароли дают разные хеши, а заранее посчитанные таблицы хешей бесполезны. bcrypt генерирует соль сам и хранит её внутри хеша.

Что значит 10 в `bcrypt.hash(password, 10)`? Стоимость: 2^{10} раундов вычислений. Чем больше, тем медленнее подбор пароля. 10 — стандартный баланс скорости и защиты.

Что внутри JWT и можно ли его подделать? Заголовок, данные (`id`, `name`, `email`, `role`, срок) и подпись. Данные читаются кем угодно, но изменить их нельзя — подпись считается с секретом `JWT_SECRET`, который есть только на сервере.

Где хранится токен на клиенте и как он попадает на сервер? В `localStorage`. При каждом защищённом запросе функция `authHeader` из `api.jsx` добавляет заголовок `Authorization: Bearer <токен>`; на сервере его читает `protect`.

Как работает выход из аккаунта? Фронт удаляет токен из `localStorage`. Сервер токены не хранит, поэтому делать на нём ничего не нужно. Старый токен сам истечёт через час.

Почему JWT, а не сессии? Сервер не хранит состояние входа — всё в подписанном токене. Фронт и сервер на разных доменах, а cookie-сессии между доменами настраивать сложнее.

Что такое middleware? Приведи примеры. Функция `(req, res, next)` между адресом и контроллером. Либо `next()`, либо сама отвечает. У нас: `protect`, `restrictTo`, `upload.single`, `validateSignup`, `validateSignin`, а также `express.json`, `cors`, `helmet`, лимитеры.

Чем 401 отличается от 403? 401 — не знаем, кто ты (нет или плохой токен). 403 — знаем, но прав нет (роль не та).

Почему роль проверяется по базе, а не по токену? Если менеджера лишили роли, его токен ещё час действителен. `protect` каждый раз берёт пользователя из базы, поэтому права пропадают сразу.

Защищена ли страница `/admin` на фронте? Только для удобства — обычного пользователя перенаправит на вход. Настоящая защита — на сервере: каждый запрос менеджера проходит `protect` и `restrictTo`.

Что такое `populate`? Замена ссылки (`ObjectId`) на сам документ из другой коллекции по полю `ref`. В корзине хранятся только id товаров; `populate` подставляет названия и цены.

Что такое `aggregate`? Многоэтапная обработка данных внутри базы: отфильтровать (`$match`), сгруппировать и посчитать (`$group`, `$sum`), развернуть массив (`$unwind`), отсортировать, ограничить. Так считается выручка и топ товаров.

Почему цена берётся из базы, а не из корзины? Данные из браузера можно подменить. `buildOrderItems` по id товара берёт цену из базы, а цену кастомной сумки считает сам по таблицам размеров и ремешков. Цена, которую прислал браузер, не используется нигде.

Что будет, если два человека одновременно купят последний товар? Списание — один атомарный запрос с условием `stock >= quantity`. База не даст уйти в минус.

Что такое вебхук и зачем он? Запрос, который Stripe сам отправляет нашему серверу после оплаты. Не зависит от браузера покупателя (вкладку могли закрыть), подписан секретом — подделать нельзя.

Почему вебхук подключён до `express.json()`? Подпись Stripe считается по сырым байтам тела. `express.json()` превратил бы их в объект, и проверка подписи не прошла бы.

Почему Stripe получает цену умноженной на 100? Stripe принимает суммы в минимальных единицах валюты — центах.

Как работает восстановление пароля и почему это безопасно? Случайный токен уходит в письме, в базе хранится только его SHA-256 хеш и срок час. Ссылка одноразовая. Ответ одинаковый для существующей и несуществующей почты.

Зачем `helmet`, `cors`, `rate-limit`? `helmet` — защитные заголовки. `cors` — разрешает фронту с другого адреса обращаться к серверу. `rate-limit` — защита от подбора паролей и спама запросами.

Как загружаются картинки? Фронт отправляет `FormData`, `multer` на сервере сохраняет файл в `uploads/` с уникальным именем, проверяет тип и размер. В базе хранится только путь, сам файл отдаёт `express.static`.

Почему корзина гостя в `localStorage`, а не на сервере? У гостя нет `user`, к которому привязать корзину на сервере. При входе гостевая корзина переносится на сервер.

Зачем контекст (Context) для корзины? Корзина нужна многим компонентам на разных уровнях. Контекст даёт к ней доступ из любого места без передачи пропсами по цепочке.

Что делает `useEffect` с пустым массивом `[]`? Выполняет код один раз после первой отрисовки — например, загрузить товары при открытии страницы.

Почему `e.preventDefault()` в формах? По умолчанию отправка формы перезагружает страницу. Мы отправляем данные сами через axios.

Что такое `.env` и почему его нет в git? Файл с секретами (пароль базы, ключи). Он в `.gitignore`, чтобы секреты не попали в публичный репозиторий. `dotenv` читает его в `process.env`.

Как стать менеджером? Зарегистрироваться обычным способом, затем администратор запускает на сервере `node seed/makeManager.js почта manager`. Через сайт роль получить нельзя.

Чем `manager` отличается от `admin`? Сейчас ничем: оба проходят `restrictTo('manager', 'admin')`. Роль `admin` заложена на будущее.

Часть 7. Слабые места — знать заранее

Лучше назвать самому, чем ждать, пока найдут. К каждому пункту — как исправить.

1. **Цены кастомных сумок записаны в двух местах** — в `CustomStudioPage.jsx` (для показа) и в `OrderController.js` (для оплаты). Если поменять цену, нужно поменять в обоих файлах. Считает деньги всегда сервер, поэтому ошибка приведёт только к тому, что на странице будет показана другая цена, но не к потере денег.
2. **Кастомные сумки вошедшего пользователя хранятся в браузере, а не в базе.** На другом компьютере их не будет. Полное решение — отдельная модель для кастомных позиций на сервере.
3. **`express-mongo-sanitize` и `xss-clean` установлены, но не подключены.** Они есть в `package.json`, но в `index.js` их нет. Либо подключить, либо удалить из зависимостей.
4. **Поиск заказов строит RegExp из введённого текста.** `new RegExp(search, 'i')` — специальные символы в поиске (например `(`) вызовут ошибку 500, а сложный шаблон может долго выполняться. Доступно только менеджеру, поэтому риск низкий. Исправление: экранировать спецсимволы перед созданием RegExp.
5. **Имя и комментарий покупателя вставляются в HTML письма без экранирования.** Если в поле имени написать HTML-код, он попадёт в письмо менеджеру. Исправление: заменять `<`, `>`, `&`, `"` на HTML-сущности.
6. **`cors()` без настроек** разрешает запросы с любого сайта. Для API с токенами в заголовке это не критично, но правильнее указать адрес своего фронта: `cors({ origin: CLIENT_URL })`.
7. **Мелочи в комментариях и логах:** в `authMiddleware.js` комментарий говорит про «7 day token», хотя токен живёт 1 час; в `index.js` сообщение о запуске всегда пишет «port 5000», даже если задан другой `PORT`.
8. **Время изготовления не вычисляется** — в конструкторе постоянный текст «2–3 weeks».
9. **Оплата на опубликованном сервере не работает без ключа Stripe** — ответ `501 Payments are not configured`. Код рабочий при наличии ключей.